

EVIDENCE — ota-manager list/ pagination wire-shape reconciliation

Revision: 1 Last modified: 2026-07-10T03:04:00Z

Task

Reconcile `clients/ota-manager/src/lib/api-client.ts`'s list/pagination TypeScript interfaces with the REAL Go server wire shapes, under the Helix Constitution anti-bluff discipline (§11.4.6 no-guessing, §11.4.108 source-truth, §11.4.115 RED → GREEN).

Confirmed defect

The client declared `DeviceList`, `ReleaseList`, `GroupList`, `TelemetryHistory`, and `DeploymentList` (all cursor-paginated list endpoints) as `{ items, total: number, cursor?: string }`. The real Go server never sends `total`, and its response-side cursor field is `next_cursor`, not `cursor` — so `total/cursor` were ALWAYS undefined at runtime and the real `next_cursor` value was completely untyped/dropped by any code that tried to read it. `RollbackList` additionally declared a fabricated `total` field for an endpoint that has no pagination at all.

Per-interface: real server shape, verified against source

Every claim below was verified by reading the cited Go source directly (not assumed from the conductor's briefing) — see the “Correction” note for the one place the briefing was wrong.

Interface	Real server shape	Verified against
DeviceList	<pre> { items, next_cursor: string \ null } </pre>	<pre> server/internal/api/ wire.go:97-100 — type DeviceList struct { Items []DeviceListItem `json:"items"`; NextCursor string `json:"next_cursor"` }(noomitempty) ReleaseList { items, next_cursor: string null } se rver/internal/api/ wire.go:159-162– same pattern GroupList { item s, next_cursor: string null } server/internal/api/ handlers_group.go:62-67–typ e GroupList struct { Items []GroupView; NextCursor string `json:"next_cursor"` } TelemetryHistory { device_id, items, next_cursor: string null } server/internal/api/ handlers_telemetry.go:30-38– type TelemetryHistory struct { DeviceID string `json:"device_id"`; Items []TelemetryEventView `json:"items"`; NextCursor *string `json:"next_cursor"` } DeploymentList { items, next_cursor: string null }– **CORRECTION, see below** server/internal/api/ wire.go:184-190 RollbackList { items }ONLY, no cursor field server/ internal/api/ handlers_recall.go:36-39– type RollbackList struct { Items []RollbackView </pre>

Interface	Real server shape	Verified against
		<pre>`json:"items"`}- confirmed no cursor field exists at all AuditLogList { items, next_cursor }- already correct in the client, NOT changed server/internal/api/ audit_wire.go:34-36`</pre>

Correction to the briefing: DeploymentList

The task briefing stated DeploymentList is {items} ONLY with no cursor field. This is **wrong** per the real server source, and per the constitution (§11.4.108: server is the source of truth) I followed the server, not the briefing:

- server/internal/api/wire.go:184-190:

```
// DeploymentList is the GET /deployments list body
// (DeploymentList schema): the
// set of active deployments. NextCursor is reserved for future
// pagination parity
// with ReleaseList (the MVP returns all active deployments in
// one page).
type DeploymentList struct {
    Items      []Deployment `json:"items"`
    NextCursor *string      `json:"next_cursor"`
}
```

- server/internal/api/handlers_deployment.go:116
(handleListDeployments) never sets NextCursor, so it is always nil today — but because the struct field has **no omitempty tag**, the JSON key is still always emitted as "next_cursor":null, never omitted. So the real wire shape IS { items, next_cursor: string | null }, with next_cursor always null in the current MVP (reserved for future pagination).
- Independent corroborating evidence found in the existing test suite: the pre-existing src/__tests__/use-telemetry-overview.test.tsx:63-69 deploymentsFixture (authored for an earlier, unrelated fix) already used next_cursor: null — NOT total/cursor — for its mocked /deployments response, consistent with this correction.

Fixed as `{ items: Deployment[]; next_cursor: string | null }` — matches the real wire bytes now and needs no client change whenever the server starts actually paginating deployments.

Files changed

1. `clients/ota-manager/src/lib/api-client.ts` — `DeviceList`, `ReleaseList`, `GroupList`, `TelemetryHistory`, `DeploymentList` changed from `{ items, total: number, cursor?: string }` to `{ items, next_cursor: string | null }`; `RollbackList` changed from `{ items, total: number }` to `{ items }` (no cursor field — genuinely none on the wire). `AuditLogList` left untouched (already correct per task instruction). Each interface got a doc comment citing the exact server file:line it was verified against.
2. `clients/ota-manager/src/__tests__/wire-shape-pagination.test.ts` — NEW guard test (compile-time, `tsc --noEmit-checked`).

No caller code needed changes — see below.

Caller audit (no fixes needed — confirmed dead-field access does not exist)

```
grep -rn '\.total\|\.cursor\|next_cursor' clients/ota-manager/src/
hooks clients/ota-manager/src/pages clients/ota-manager/src/
components (note: src/pages does not exist in this project; pages live
under src/features, which was also checked) surfaced only:
```

- `filters?.cursor` in `use-audit.ts`, `use-deployments.ts`, `use-devices.ts` (×2), `use-releases.ts` — all legitimate **request-side** query-param reads off the `*Filter` interfaces (`DeviceListFilter.cursor`, `ReleaseFilter.cursor`, `TelemetryFilter.cursor`, `AuditFilter.cursor`), which correctly map to the server's accepted `?cursor=` query parameter (confirmed server-side: `c.Query("cursor")` in `handlers_device.go:114`, `handlers_release.go:97`, `handlers_group.go:28`, `handlers_telemetry.go:100`, `handlers_audit.go:137`). These were NOT touched — they are correct and out of scope (request params, not response fields).
- `pagination?.total` in `components/data-table/data-table.tsx:115` — this is the `DataTable` component's own local `PaginationState.total` prop type (an unrelated, purely-presentational prop a caller would have to pass explicitly). Confirmed via `grep -n "pagination" src/`

features/**/*.tsx that **no page currently wires a pagination= prop** sourced from any of the six response types fixed here, so this is dead code the component itself owns, not a caller reading a removed response field.

- stats.total in deployment-detail-page.tsx (found via a broader repo grep) is DeviceStatusBreakdownProps.stats: { succeeded, failed, pending, total } — a wholly separate, locally-defined prop type, unrelated to DeploymentList. Confirmed untouched and unaffected.
- deploymentsQuery.data?.items.length in useTelemetryOverview.ts:102 reads only .items (never .total/.cursor) — confirmed this still type-checks after the fix (see tsc run below), satisfying the task's explicit preserve-behavior constraint.

No interface consumer constructs a typed literal of any of the six response interfaces (grep -rn ': DeviceList\b\|: ReleaseList\b\|: GroupList\b\|: TelemetryHistory\b\|: DeploymentList\b\|: RollbackList\b' src — zero hits), so removing total/cursor broke no existing call site.

\$11.4.115 RED → GREEN evidence

The guard test (wire-shape-pagination.test.ts) type-checks six real-shaped fixtures against the six interfaces, plus two @ts-expect-error assertions that the OLD fabricated fields (total on DeviceList, next_cursor on RollbackList) are rejected if reintroduced. tsc --noEmit is the oracle — the test body itself is a one-line runtime no-op (expect(true).toBe(true)) so a future deletion of the type-level guard cannot masquerade as “still passing” under vitest's own file-must-have-tests check.

RED (pre-fix — interfaces manually reverted to the OLD shape, guard test kept)

Command: npx tsc --noEmit with api-client.ts's six interfaces reverted to { items, total: number, cursor?: string } (RollbackList: { items, total: number }), guard test unchanged:

```
src/__tests__/wire-shape-pagination.test.ts(52,47): error TS2353:
Object literal may only specify known properties, and
'next_cursor' does not exist in type 'DeviceList'.
src/__tests__/wire-shape-pagination.test.ts(53,49): error TS2353:
```

```
Object literal may only specify known properties, and
'next_cursor' does not exist in type 'ReleaseList'.
src/__tests__/wire-shape-pagination.test.ts(54,45): error TS2353:
Object literal may only specify known properties, and
'next_cursor' does not exist in type 'GroupList'.
src/__tests__/wire-shape-pagination.test.ts(55,59): error TS2353:
Object literal may only specify known properties, and
'next_cursor' does not exist in type 'TelemetryHistory'.
src/__tests__/wire-shape-pagination.test.ts(56,55): error TS2353:
Object literal may only specify known properties, and
'next_cursor' does not exist in type 'DeploymentList'.
src/__tests__/wire-shape-pagination.test.ts(57,9): error TS2741:
Property 'total' is missing in type '{ items: never[]; }' but
required in type 'RollbackList'.
```

Six real errors, one per interface — the guard genuinely fails on the old (wrong) shape. The file was then restored to the fixed state (verified via a byte-for-byte copy taken before the revert).

GREEN (post-fix — real interfaces restored)

Command: `npx tsc --noEmit`

Output: **empty, exit code 0**. All six real-shaped fixture assignments type-check; both `@ts-expect-error` assertions correctly suppress their expected errors (confirmed no “unused `@ts-expect-error` directive” error was emitted, which would fire if the fabricated fields had stopped erroring).

Verification commands run (2026-07-10, ota-manager repo)

`npx tsc --noEmit`

```
$ npx tsc --noEmit
(no output)
$ echo $?
0
```

Exit code 0 — clean.

npx vitest run

```
RUN v4.1.9 /home/milos/Factory/projects/tools_and_research/helix_ota/clients/ota-manager
```

```
Test Files  12 passed (12)
  Tests     50 passed (50)
  Start at  03:04:35
  Duration  7.97s (transform 264ms, setup 349ms, import 1.24s,
tests 903ms, environment 4.39s)
```

12 test files / 50 tests, all passing — includes the 11 pre-existing files plus the new wire-shape-pagination.test.ts.

Dead-field-access grep

```
$ grep -rn '\.total\|\.cursor' clients/ota-manager/src/hooks
clients/ota-manager/src/components
src/hooks/use-audit.ts:25:      if (filters?.cursor) params.cursor
= filters.cursor;
src/hooks/use-deployments.ts:37:      if (filters?.cursor)
params.cursor = filters.cursor;
src/hooks/use-devices.ts:65:      if (filters?.cursor)
params.cursor = filters.cursor;
src/hooks/use-devices.ts:82:      if (filters?.cursor)
params.cursor = filters.cursor;
src/components/data-table/data-table.tsx:115:    rowCount:
pagination?.total,
src/hooks/use-releases.ts:32:      if (filters?.cursor)
params.cursor = filters.cursor;
```

All six hits are legitimate (request-side filter params / unrelated local component prop), per the caller audit above. `src/pages` does not exist in this project (checked; confirmed no matches to omit).

Preserved-behavior confirmation

`deploymentsQuery.data?.items.length` (`useTelemetryOverview.ts:102`) type-checks unchanged after the fix — confirmed by the clean `tsc --noEmit` run above (this line would be the first to break under `strict/noUncheckedIndexedAccess` if items had been touched; it was not).

Scope discipline note

TelemetryHistory's real wire shape also carries a `device_id: string` field (`handlers_telemetry.go:35`) that the client interface has never declared. This is a genuine, truthful gap or improvement, but is OUT of the requested scope (the confirmed defect and this task were specifically about the `total/cursor → next_cursor` pagination-field mismatch) and no caller reads `.device_id` on this response today, so it was deliberately NOT added here to avoid unrequested scope creep. Flagging for a future, separately scoped item if the caller ever needs it.